SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA0306	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Y SRAVAN KUMAR	**Reg. No.:**	192472289
Section / Batch:	Slot C 2024	**Date:**	15-08-2026

Code of Conduct

I Y SRAVAN KUMAR (192472289) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: SRAVAN

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	15	15	18	15	12	75	75
2	14	16	17	15	13	75	
3	15	14	16	17	13	75	
4	16	15	15	16	13	75	
5	15	16	17	14	13	75	

Faculty In-charge	Course Coordinator	Program Director
Signature: DR.T.Kumaragurubaran_	Signature: _____	Signature: _____
Date: 15-08-2026	Date: _____	Date: _____

OUTPUT

HMM.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/HMM.py (3.12.3)

File Edit Format Run Options Window Help

```
# ------------------------------------------------------------
# HIDDEN MARKOV MODEL (HMM)
# POS Tagging without NLP Libraries
# ------------------------------------------------------------

# Training Corpus
corpus = [
    ("The boy eats rice", "DT NN VBZ NN"),
    ("The girl drinks milk", "DT NN VBZ NN"),
    ("A cat drinks milk", "DT NN VBZ NN"),
    ("The dog chases cat", "DT NN VBZ NN"),
    ("A teacher teaches students", "DT NN VBZ NNS"),
    ("Students study English", "NNS VBP NN"),
    ("Birds fly high", "NNS VBP RB"),
    ("Children play games", "NNS VBP NNS")
]

# Dictionaries
emission_count = {}
transition_count = {}
tag_count = {}
start_count = {}

# ------------------------------------------------------------
# TASK 1: Separate Words and POS Tags
# ------------------------------------------------------------

print("============================================")
print("TASK 1: Separate Words and POS Tags")
print("============================================")

for sentence, tags in corpus:

    words = sentence.split()
    tag_list = tags.split()

    print("\nSentence :", words)
    print("Tags     :", tag_list)

    # Start tag count
    first_tag = tag_list[0]
    start_count[first_tag] = start_count.get(first_tag, 0) + 1

    # Emission count
    for word, tag in zip(words, tag_list):
```

IDLE Shell 3.12.3

File Edit Shell Debug Options Window Help

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 1
Type "help", "copyright", "credits" or "license()"
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Pr
============================================
TASK 1: Separate Words and POS Tags
============================================

Sentence : ['The', 'boy', 'eats', 'rice']
Tags     : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['The', 'girl', 'drinks', 'milk']
Tags     : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['A', 'cat', 'drinks', 'milk']
Tags     : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['The', 'dog', 'chases', 'cat']
Tags     : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['A', 'teacher', 'teaches', 'students']
Tags     : ['DT', 'NN', 'VBZ', 'NNS']

Sentence : ['Students', 'study', 'English']
Tags     : ['NNS', 'VBP', 'NN']

Sentence : ['Birds', 'fly', 'high']
Tags     : ['NNS', 'VBP', 'RB']

Sentence : ['Children', 'play', 'games']
Tags     : ['NNS', 'VBP', 'NNS']

============================================
TASK 2: Emission Probability
============================================

 DT
The : 0.6
A : 0.4

 NN
boy : 0.1
rice : 0.1
girl : 0.1
milk : 0.2
```

CODE:

```
# ------------------------------
# HIDDEN MARKOV MODEL (HMM)
# POS Tagging without NLP Libraries
# ------------------------------
# Training Corpus
corpus = [
("The boy eats rice", "DT NN VBZ NN"),
("The girl drinks milk", "DT NN VBZ NN"),
("A cat drinks milk", "DT NN VBZ NN"),
("The dog chases cat", "DT NN VBZ NN"),
("A teacher teaches students", "DT NN VBZ NNS"),
("Students study English", "NNS VBP NN"),
("Birds fly high", "NNS VBP RB"),
("Children play games", "NNS VBP NNS")
]
# Dictionaries
emission_count = {}
transition_count = {}
tag_count = {}
start_count = {}
print("============================================")
print("TASK 1: Separate Words and POS Tags")
print("============================================")
# Step 1: Split words and tags
for sentence, tags in corpus:
    words = sentence.split()
    tag_list = tags.split()

    print("\nSentence :", words)
    print("Tags     :", tag_list)

    # Count Start Tags
    first_tag = tag_list[0]
    start_count[first_tag] = start_count.get(first_tag, 0) + 1

    # Count Emission
    for word, tag in zip(words, tag_list):

        tag_count[tag] = tag_count.get(tag, 0) + 1

        if tag not in emission_count:
            emission_count[tag] = {}

        emission_count[tag][word] = emission_count[tag].get(word, 0) + 1

    # Count Transition
    for i in range(len(tag_list)-1):

        current = tag_list[i]
        nxt = tag_list[i+1]
```

```
        if current not in transition_count:
            transition_count[current] = {}

        transition_count[current][nxt] = transition_count[current].get(nxt, 0) + 1


# ---------------------------------------
# TASK 2 : Emission Probability
# ---------------------------------------

print("\n============================================")
print("TASK 2 : Emission Probability")
print("============================================")

emission = {}

for tag in emission_count:

    emission[tag] = {}

    print("\n", tag)

    for word in emission_count[tag]:

        probability = emission_count[tag][word] / tag_count[tag]

        emission[tag][word] = probability

        print(word, ":", round(probability,3))


# ---------------------------------------
# TASK 3 : Transition Probability
# ---------------------------------------
print("\n============================================")
print("TASK 3 : Transition Probability")
print("============================================")
transition = {}
for tag in transition_count:
    transition[tag] = {}
total = sum(transition_count[tag].values())
for nxt in transition_count[tag]:
    probability = transition_count[tag][nxt] / total
transition[tag][nxt] = probability
print(tag,"->",nxt,"=",round(probability,3))
# ---------------------------------------
# Start Probability
# ---------------------------------------
start_probability = {}
total_sentences = len(corpus)
for tag in start_count:
```

```
    start_probability[tag] = start_count[tag] / total_sentences
# --------------------------------------
# TASK 4 : Hidden Markov Model
# --------------------------------------
print("\n============================================")
print("TASK 4 : Hidden Markov Model")
print("============================================")
print("\nStart Probability")
print(start_probability)
print("\nEmission Probability")
print(emission)
print("\nTransition Probability")
print(transition)
# --------------------------------------
# TASK 5 : Viterbi Algorithm
# --------------------------------------

print("\n============================================")
print("TASK 5 : Viterbi Algorithm")
print("============================================")

sentence = input("\nEnter a sentence : ").split()

tags = list(emission.keys())

V = {}
path = {}

# Initialization

first_word = sentence[0]

for tag in tags:

    emit = emission[tag].get(first_word,0)

    start = start_probability.get(tag,0)

    if emit > 0 and start > 0:

        V[tag] = start * emit

        path[tag] = [tag]

# Recursion

for word in sentence[1:]:

    newV = {}
    newPath = {}

    for current_tag in tags:
```

```
        emit = emission[current_tag].get(word,0)

        if emit == 0:
            continue

        best_probability = 0
        best_previous = None

        for previous_tag in V:

            trans = transition.get(previous_tag,{}).get(current_tag,0)

            probability = V[previous_tag] * trans * emit

            if probability > best_probability:

                best_probability = probability
best_previous = previous_tag
if best_previous is not None:
    newV[current_tag] = best_probability
newPath[current_tag] = path[best_previous] + [current_tag]
V = newV
path = newPath
# ---------------------------------------
# TASK 6 : Prediction
# ---------------------------------------
print("\n============================================")
print("TASK 6 : Predicted POS Tags")
print("============================================")
if len(V)==0:
    print("No valid tag sequence found.")
else:
    best_tag = max(V,key=V.get)
predicted = path[best_tag]
print("\nSentence :", " ".join(sentence))
print("\nPredicted Tags :")
for word,tag in zip(sentence,predicted):
    print(word,"/",tag)
print("\nTag Sequence :")
print(" ".join(predicted))
```

output:

```
==========================================
TASK 1: Separate Words and POS Tags
==========================================

Sentence : ['The', 'boy', 'eats', 'rice']
Tags     : ['DT', 'NN', 'VBZ', 'NN']
```

Sentence : ['The', 'girl', 'drinks', 'milk']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['A', 'cat', 'drinks', 'milk']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['The', 'dog', 'chases', 'cat']
Tags : ['DT', 'NN', 'VBZ', 'NN']

Sentence : ['A', 'teacher', 'teaches', 'students']
Tags : ['DT', 'NN', 'VBZ', 'NNS']

Sentence : ['Students', 'study', 'English']
Tags : ['NNS', 'VBP', 'NN']

Sentence : ['Birds', 'fly', 'high']
Tags : ['NNS', 'VBP', 'RB']

Sentence : ['Children', 'play', 'games']
Tags : ['NNS', 'VBP', 'NNS']

==========
TASK 2 : Emission Probability
==========

DT
The : 0.6
A : 0.4

NN
boy : 0.1
rice : 0.1
girl : 0.1
milk : 0.2
cat : 0.2
dog : 0.1
teacher : 0.1
English : 0.1

VBZ
eats : 0.2
drinks : 0.4
chases : 0.2
teaches : 0.2

NNS
students : 0.2
Students : 0.2
Birds : 0.2
Children : 0.2
games : 0.2

VBP
study : 0.333
fly : 0.333
play : 0.333

RB
high : 1.0

==========
TASK 3 : Transition Probability
==========
VBP -> NNS = 0.333

==========
TASK 4 : Hidden Markov Model
==========

Start Probability
{'DT': 0.625, 'NNS': 0.375}

Emission Probability
{'DT': {'The': 0.6, 'A': 0.4}, 'NN': {'boy': 0.1, 'rice': 0.1, 'girl': 0.1, 'milk': 0.2, 'cat': 0.2, 'dog': 0.1, 'teacher': 0.1, 'English': 0.1}, 'VBZ': {'eats': 0.2, 'drinks': 0.4, 'chases': 0.2, 'teaches': 0.2}, 'NNS': {'students': 0.2, 'Students': 0.2, 'Birds': 0.2, 'Children': 0.2, 'games': 0.2}, 'VBP': {'study': 0.3333333333333333, 'fly': 0.3333333333333333, 'play': 0.3333333333333333}, 'RB': {'high': 1.0}}

Transition Probability
{'DT': {}, 'NN': {}, 'VBZ': {}, 'NNS': {}, 'VBP': {'NNS': 0.3333333333333333}}

==========
TASK 5 : Viterbi Algorithm
==========

Enter a sentence : A cat drinks milk.

==========
TASK 6 : Predicted POS Tags
==========

Sentence : A cat drinks milk.

Predicted Tags :
A / DT
cat / RB

Tag Sequence :
DT RB
>>>